

- 算法特性

算法具有的重要特征

- 有限性
- 确定性
- 可行性
- 输入性
- 输出性

- 主定理

主定理：设 $a \geq 1, b > 1$ 为常数， $f(n)$ 为函数， $T(n)$ 为非负正数，且： $T(n) = aT(n/b) + f(n)$

则有以下结果：

- (1) 若 $f(n) = O(n^{\log_b a - \epsilon})$ ， $\epsilon > 0$ ，那么 $T(n) = \Theta(n^{\log_b a})$
- (2) 若 $f(n) = \Theta(n^{\log_b a})$ ，那么 $T(n) = \Theta(n^{\log_b a} \log n)$
- (3) 若 $f(n) = \Omega(n^{\log_b a + \epsilon})$ ， $\epsilon > 0$ ，且对于某个常数 $c < 1$ 和所有充分大的 n ，有 $af(n/b) \leq cf(n)$ ，那么 $T(n) = \Theta(f(n))$

主定理应用举例：

1. 求解递推方程 $T(n)=9T(n/3)+n$

$a=9, b=3, f(n)=n, n^{\log_3 9}=n^2, f(n)=O(n^{\log_3 9-1}), \epsilon=1, T(n)=\Theta(n^2)$

1. 求解递推方程 $T(n)=T(2n/3)+1$

$a=1, b=3/2, f(n)=1, n^{\log_{3/2} 1}=n^0=1, f(n)=1$, 相当于主定理的第二种情况,
 $T(n)=\Theta(n^0 \log n)=\Theta(\log n)$

1. 求解递推方程 $T(n)=3T(n/4)+n \log n$

$a=3, b=4, f(n)=n \log n$, 那么 $n \log n = \Omega(n^{\log_4 3 + \epsilon}) = \Omega(n^{0.793 + \epsilon}), \epsilon \approx 0.2$

此外, 要使 $af(n/b) \leq cf(n)$ 成立, 带入 $f(n)=n \log n$, 得到:

$3/4 \log n / 4 \leq c n \log n$, 显然只要 $c \geq 3/4$ 即可。相当于主定理第三种情况

$T(n)=\Theta(f(n))=\Theta(n \log n)$

分治法求解矩阵相乘原理

3. Strassen算法

简单的分治算法需8次乘法和 4次加减法, 本算法需要7次乘法和 18次加减法。

该算法吸引我们的不是这个原因, 而在于当 n 很大时所表现的卓越效率

算法将原来的8次递归变成了P1~P7的7次递归, 同理有:

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 7T(n/2) + \Theta(n^2) & n > 1 \end{cases}$$

根据主定理有 $T(n)=\Theta(n^{\log_b a})=\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807}) < \Theta(n^3)$, Strassen算法的渐进复杂性低于简单的分治过程

- 空间复杂度

算法空间复杂度

算法的空间复杂度是指算法在执行过程中所需的存储空间量级。一个算法的存储量包括形参所占空间和临时变量所占空间。在对算法进行存储空间分析时，只考察临时变量所占空间。

例如，有以下算法，其中临时空间为变量*i*、*max i*占用的空间。所以，空间复杂度是对一个算法在运行过程中临时占用的存储空间大小的量度，一般也作为问题规模*n*的函数，以数量级形式给出，记作：

$$S(n)=O(g(n))、\Omega(g(n))或\Theta(g(n))$$

其中渐进符号的含义与时间复杂度中的含义相同。

- 递归的条件

递归需满足的条件

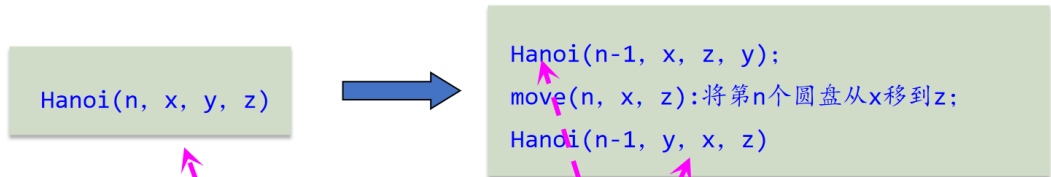
一般来说，能够用递归解决的问题应该满足以下三个条件：

- 需要解决的问题可以转化为一个或多个子问题来求解，而这些子问题的求解方法与原问题完全相同，只是在数量规模上不同。
- 递归调用的次数必须是有限的。
- 必须有结束递归的条件来终止递归。

- 汉诺塔

何时使用递归

设 $\text{Hanoi}(n, x, y, z)$ 表示将 n 个盘片从 x 通过 y 移动到 z 上，递归分解的过程是：



“大问题”转化为若干个“小问题”求解

- 递归转非递归

递归转非递归

把递归算法**转化**为非递归算法有如下两种基本方法：

- (1) 直接用循环结构的算法替代递归算法。
- (2) 用栈模拟系统的运行过程，通过分析只保存必须保存的信息，从而用非递归算法替代递归算法。

第(1)种是直接转化法，不需要使用栈。第(2)种是间接转化法，需要使用栈。

```

int n;

void qsort(int l, int r) {
    if (l >= r) return;

    int p=rand()%(r-l+1)+l;
    swap(a[l],a[p]);

    int i = l + 1, j = r;
    while (i <= j) {
        while (i <= j && a[i] < a[l]) i++;
        while (i <= j && a[j] > a[l]) j--;
        if (i <= j) {
            swap(a[i], a[j]);
            i++;
            j--;
        }
    }
    swap(a[l], a[j]);

    qsort(l, j);
    qsort(i, r);
}

```

- 快速排序
- dp的条件

7.1 动态规划的基本原理

能采用动态规划求解的问题的一般要具有3个性质：

- **最优性原理**：如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构，即满足最优性原理。
- **无后效性**：即某阶段状态一旦确定，就不受这个状态以后决策的影响。也就是说，某状态以后的过程不会影响以前的状态，只与当前状态有关。
- **有重叠子问题**：即子问题之间是不独立的，一个子问题在下一阶段决策中可能被多次使用到。（该性质并不是动态规划适用的必要条件，但是如果没有这条性质，动态规划算法同其他算法相比就不具备优势）。

- 问题的解空间
 - 一个复杂问题的解决方案是由若干个小的决策步骤组成的决策序列，解决一个问题的所有可能的决策序列构成该问题的**解空间**。
 - 应用回溯法求解问题时，首先应该明确问题的解空间。解空间中满足约束条件的决策序列称为**可行解**。一般来说，解任何问题都有一个目标，在约束条件下使目标达到最优的可行解称为该问题的**最优解**。
 - 解空间的类型：
 - 当所给的问题是从n个元素的集合S中找出满足某种性质的子集时，相应的解空间树称为**子集树**。
 - 当所给的问题是确定n个元素满足某种性质的排列时，相应的解空间树称为**排列树**。
- 剪枝函数：
 - 用**约束函数**在扩展结点处剪除不满足约束的子树；用**限界函数**剪去得不到问题解或最优解的子树。
- 回溯法的步骤

归纳起来，用回溯法解题的**一般步骤**如下：

- ① 针对所给问题，确定问题的解空间树，问题的解空间树应至少包含问题的一个（最优）解。
- ② 确定结点的扩展搜索规则。
- ③ 以**深度优先方式**搜索解空间树，并在搜索过程中可以采用剪枝函数来避免无效搜索。

- A*算法

A-star算法

下面是一个利用分支限界法解决A*问题的算法描述：

初始化： 将起始点添加到一个优先级队列中，队列中的每个节点包含一个区域、从起始点到该区域的实际移动代价 $g(n)$ ，以及启发式函数估计的移动代价 $h(n)$ 。初始时，队列只包含起始点，并且 $g(n)$ 为0。

循环： 从优先级队列中选择具有最低 $f(n) = g(n) + h(n)$ 值的节点，这个节点称为当前节点。如果当前节点是目标节点，停止循环。

代价函数： $f(n) = g(n) + h(n)$ ，方法效率受代价函数影响

扩展： 对当前节点进行扩展，即将当前节点的可通行相邻区域添加到队列中。对于每个相邻区域，计算新的 $g(n)$ 值（即从起始点到当前相邻区域的实际移动代价），并计算 $h(n)$ 值（从当前相邻区域到目标点的启发式估计）。然后计算 $f(n)$ 值，将相邻区域添加到队列中。

- 贪心算法基本要素：

6.1 优化原理与贪心算法的基本要素

- **贪心算法的基本要素：**

1. 贪心选择性质：每一步选择最优解的策略。
2. 最优子结构：问题的最优解可以通过子问题的最优解推导得到。

- P问题

02 P类和NP类问题

P类问题

用确定性图灵机、以**多项式时间**内可解的问题，称为**P 类问题**。P是多项式时间 Polynomial time的缩写。

更具体地说，它们是指对于某个常数 k ，可以在 $O(n^k)$ 时间内求解的问题，其中 **n 是问题输入的大小**

理论上，一个问题只要能够在多项式时间之内解决，不管是线性时间或者 n 次方多项式的时间内，我们都通通称之为P问题。

- NP问题

02 P类和NP类问题

NP类问题

用非确定性图灵机、以多项式时间内可解的问题，称为 **NP 问题**，
NP 指非确定性图灵机上的、具有多项式算法的问题集合，这里N是指不确定的意思。

$$P \subseteq NP$$

与P问题相比：

P 问题是指能在多项式时间复杂度内可以找到解的问题，

而 NP 问题是指给定一个解，能在多项式时间复杂度内可以验证其是否正确的问题

- NPC问题

03 NPC问题

NP-completeness(NPC问题、NP完全问题)

1.它是一个**NP问题**

2.所有的NP问题都可以用多项式时间**归约**到它

定义：

设 $L_1 \subseteq \Sigma_1^*$ ， $L_2 \subseteq \Sigma_2^*$ 为两个语言，若存在映射 $f: \Sigma_1^* \rightarrow \Sigma_2^*$ ，使得：

(1) 存在多项式时间界的确定图灵机求解 f ；

(2) $\forall x \in \Sigma_1^*$ ， $x \in L_1 \Leftrightarrow f(x) \in L_2$ 则称 L_1 可以多项式归约为 L_2 ，记为 $L_1 \propto L_2$ 。例如，

有向哈密尔顿回路问题可多项式归约为旅行商问题

多项式归约具有以下性质，每个问题Q都是NPC问题：

$Q_1 \in P$ ，若 $Q_2 \propto Q_1$ ，则 $Q_2 \in P$ 。

若 $Q_1 \propto Q_2$ ，且 $Q_2 \propto Q_3$ ，则 $Q_1 \propto Q_3$ 。

- NP-hard问题

03 NPC问题

NP-hard问题

如果所有NP问题都可以多项式时间归约到某个问题，则称该问题为NP-hard问题。

$$L' \leq_p L \text{ for every } L' \in NP$$

其满足NPC问题定义的第二条但不一定要满足第一条(NP-hard问题不一定是NP问题)

下围棋就是NP-hard问题，但不是NP问题，我们要在一个残局上找一个必胜下法，告诉我们下一步下在哪里。显然，我们找不这个解，而且更难的是，就算有人给我了一个解，我们也无法在P时间内判断它是不是正确的。